

Milestone 2 — Build Your Own Ground

A home-built `GridWorld` environment where a borrowed PPO brain learns to reach the goal.

Status: ✓ COMPLETE — “Done when” satisfied

Sprint: M2

Date: 2026-07-08

Author: Phillip

Branch: M2-Implementation

M2 is the first milestone where we **build a Ground of our own** instead of borrowing one. We wrote `GridWorldEnv`, a 5x5 walk-to-the-goal world that obeys the exact same Gymnasium contract (`reset()` / `step()`) as the borrowed `CartPole`. Because the contract never changed, the **same PPO brain from M1 plugged straight in and learned** — going from a random-walk baseline of about `-0.3` reward/episode to `+0.93`, finding the theoretically optimal 8-step path and reaching the goal in 20/20 evaluation episodes. That swap — new Ground, unchanged brain — is the whole point of the project, and M2 proves it end-to-end.

5 / 5

BRICKS DONE

9 / 9

TESTS PASSING

+0.93

BEST MEAN REWARD

20 / 20

GOALS REACHED

8.0

STEPS TO GOAL (OPT.)

0

CONTRACT CHANGES

1 Accomplishments — the 5 bricks

#	BRICK	FILE	DONE WHEN...	STATUS
1	Build the <code>GridWorld</code> env 5x5 grid, <code>Discrete(4)</code> moves, <code>Box</code> (row,col) obs, <code>-0.01/step</code> , <code>+1.0</code> goal.	<code>src/gametrainer/gridworld.py</code>	SB3 <code>check_env</code> passes; <code>reset</code> →2-tuple, <code>step</code> →5-tuple.	DONE
2	Lock the contract with tests 9 tests: <code>reset/step</code> shapes, goal terminates, cap truncates, all 4 moves, walls, step cost.	<code>tests/test_gridworld.py</code>	<code>pytest</code> green.	DONE
3	Random-agent runner (baseline) 20 random episodes; prints the low/negative baseline PPO must beat.	<code>scripts/run_gridworld.py</code>	Runs; prints a low/negative baseline score.	DONE
4	PPO learns to reach the goal PPO <code>MlpPolicy</code> , 25k steps, Eval + Checkpoint callbacks, printed PASS/FAIL verdict.	<code>scripts/train_gridworld.py</code>	Mean reward clearly beats baseline → M2 “Done when”.	DONE
5	Wire into TUI + changelog Menu items [3] Run <code>GridWorld</code> and [4] Train <code>GridWorld</code> ; M2 changelog entry.	<code>tui.py</code> · <code>docs/CHANGELOG.md</code>	New menu items launch the scripts.	DONE

2 The result — PPO learned the optimal path

Evaluation reward vs. training steps



Source: `logs/gridworld/evaluations.npz` (13 evals, deterministic, 10 episodes each). Flat at `-1.0` (always hits the 100-step cap, never finds the goal) until a clean step-change at `~12k` steps.

Verdict vs. definition of done

Random baseline	≈ -0.30
Pass threshold	$\geq +0.50$
PPO best mean reward	+0.93
Goal reached (greedy)	20 / 20
Mean steps to finish	8.0 (optimal)



PASS. Reward is **+1.23** above the random baseline and clears the bar by a wide margin. The agent reaches the goal consistently via the shortest route — **M2** “Done when” is satisfied.

3 Retrospective — what went well & what didn't

What went well

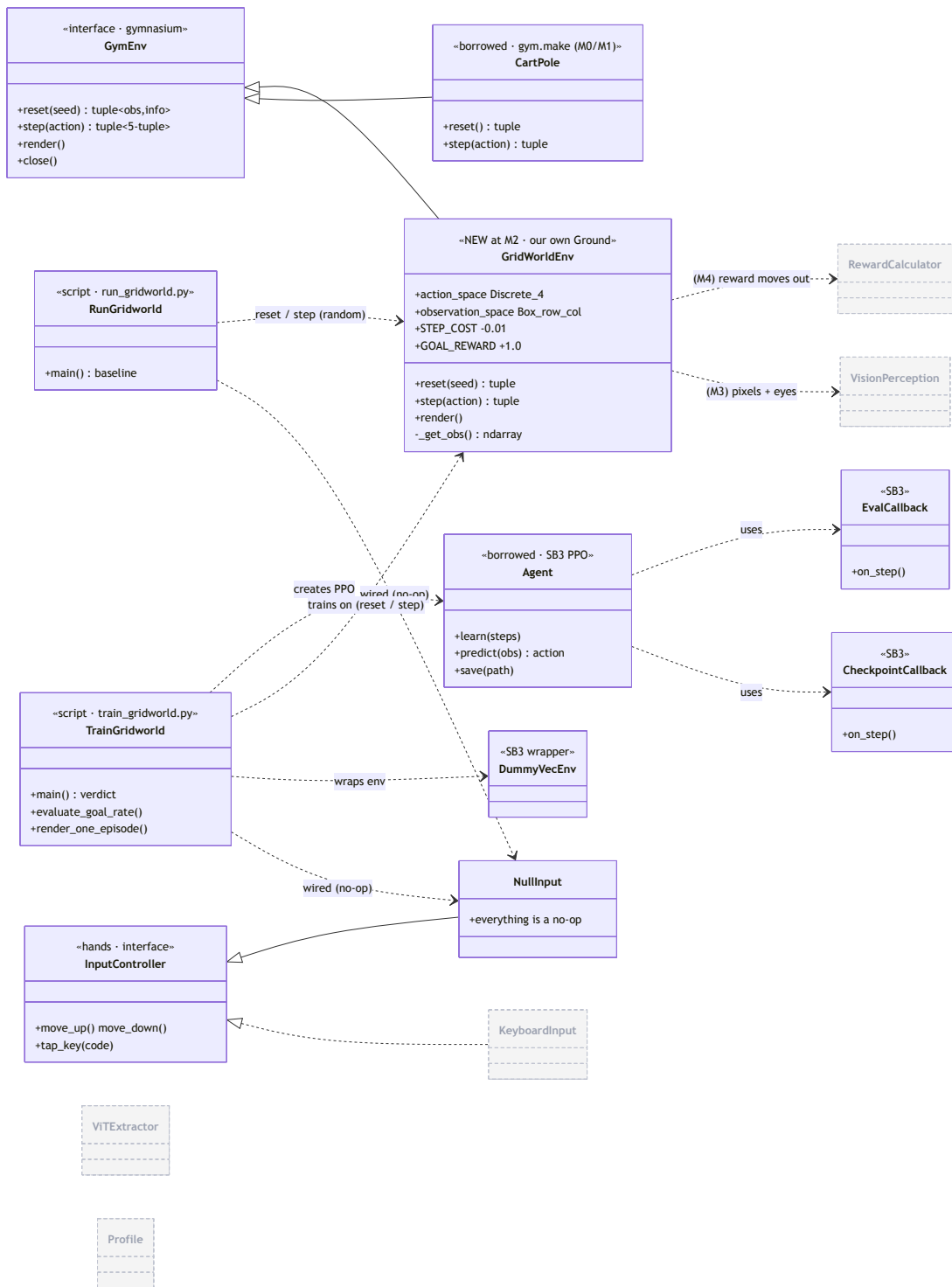
- **The contract held.** Swapping the Ground (CartPole → our GridWorld) needed **zero changes** to the PPO brain or the callbacks — exactly the design promise.
- **Clean, decisive learning.** A textbook step-change: the agent jumped from never solving it to the **optimal 8-step path** and stayed there (no wobble, no regression).
- **Tests came first and stuck.** 9 contract tests (Brick 2) lock `reset / step` shapes, so any future edit that breaks the contract turns `pytest` red immediately.
- **Consistency with M1.** Scripts mirror `run_cartpole.py / train_cartpole.py` one-for-one, so the codebase reads as one system.
- **Scope discipline.** No Profile, no ViT, no C++ — we built only the brick in front of us.

What didn't go well / friction

- **Baseline is hardcoded.** `RANDOM_BASELINE = -0.3` is a documented approximation; the real random score drifts run-to-run (~ -0.2 to -0.5). The verdict should compute it live, not assume it.
- **The “2x baseline” rule broke.** M1's pass rule fails on a **negative** baseline, so we switched to an absolute `+0.5` bar — a reasonable but hand-picked threshold.
- **An extra eval at 26k.** With `n_steps=2048`, step counts don't divide evenly, leaving a stray 13th eval past the 25k budget — harmless, but untidy.
- **NullInput is dead weight here.** It's wired in for architectural visibility but does nothing; a reader could mistake it for real behavior.
- **No CI yet.** Tests pass locally but aren't enforced on push, and this work sits on an unmerged branch.

4 Current software architecture (UML class diagram)

Solid boxes are **built & wired today**; greyed dashed boxes are **planned** for later milestones. New at M2: `GridWorldEnv` (our own Ground) and the two `GridWorld` scripts. The PPO `Agent` and SB3 callbacks are unchanged from M1 — they simply point at the new environment.



 Built & wired (M0–M2)
 Planned (M3+)
 <|-- inheritance
 ..> depends on / uses



The one idea to take away: `GridWorldEnv` and `CartPole` both sit under the same `GymEnv` contract. Because of that shared shape, the borrowed `Agent` (PPO) trained on our brand-new world **without a single change to itself**. Build the Ground, borrow the Brain — and they still fit.

5 Scope discipline — what M2 deliberately did *not* build

X No reward abstraction

Reward logic lives **directly inside** `GridWorldEnv`. No `Profile` / `RewardCalculator` — that's **M4** (config-only swapping).

X No eyes / pixels

Observation is two numbers `(row, col)`, not an image. No ViT, no `VisionPerception` — that's **M3** ("add the eyes").

X No real input / C++

`GridWorld` is programmatic. No screen capture and no real key presses via the `C++ cli` — that's **M5**.

6 Next steps

Immediate (close out M2)

- **Merge** `M2-Implementation` → `main` via PR (mirrors the M1 flow).
- **Compute the baseline live** in the verdict instead of the hardcoded `-0.3`, so PASS/FAIL is self-contained.
- **Add CI** to run `pytest` on push, protecting the Gymnasium contract automatically.
- Optional: tidy the stray 13th eval by aligning `eval_freq` / `n_steps` to the 25k budget.

Next milestone → M3: "Add the Eyes"

- Swap the numeric `(row, col)` observation for a **picture** of the grid.
- Introduce the **ViT backbone** (pretrained "eyes") + `CnnPolicy` / feature extractor.
- **Contract stays identical** — only the observation space changes; the brain still plugs in.
- First point where the **AMD GPU** matters (earlier phases were CPU-only).